

포팅 매뉴얼

Project Skill Stack Version

BE

Skill	Version
Java	17
SpringBoot	3.5.3
AWS SDK	2.32.9
MySQL	8.0.42
JWT	0.12.6

FE

Skill	Version
Vue.js	3

EC2 포트 번호

Skill	Port
Backend	8081
Nginx	80
MySQL	3306
Jenkins	8080

외부 프로그램

- LiveKit

환경 변수 (.env)

▼ Backend-API-SERVER

c204-be-api/
└── .env # 파일 위치

```
SERVER_PORT=8081

MYSQL_LOCAL_HOST=localhost
MYSQL_LOCAL_PORT=3307
MYSQL_LOCAL_USERNAME=root
MYSQL_LOCAL_PASSWORD=1234

MYSQL_TEST_HOST=i13C204.p.ssafy.io
MYSQL_TEST_PORT=3307
MYSQL_TEST_USERNAME=root
MYSQL_TEST_PASSWORD=1q2w3e4r!

MYSQL_PROD_HOST=i13C204.p.ssafy.io
MYSQL_PROD_PORT=3306
```

▼ Frontend

fe/
└── .env.local # 파일 위치

```
# local
VITE_KAKAO_JAVASCRIPT_KEY=060685617d8676
e3f65e0776e265c16c
VITE_API_BASE_URL=http://localhost:8081
VITE_WEB_RTC_BASE_URL=wss://lol-demh9a97.liv
ekit.cloud
```

```
# prod
VITE_KAKAO_JAVASCRIPT_KEY=060685617d8676
e3f65e0776e265c16c
VITE_API_BASE_URL=https://i13c204.p.ssafy.io
```

```
MYSQL_PROD_USERNAME=root
MYSQL_PROD_PASSWORD=1q2w3e4r!
```

```
MYSQL_DATABASE=lol
```

```
JWT_SECRET_KEY=jewipofjpvnxcnvioladhfiouwqej
hofasdfjwqeopfjpeowqjfpovnmxcznmznsdlfjweqf
peowqjfk
```

```
JWT_ACCESS_TOKEN_EXPIRATION_MILLIS=12960
00000
```

```
JWT_REFRESH_TOKEN_EXPIRATION_DAYS=14
```

```
KAKAO_CLIENT_ID=3fca693b8d122c78190a536bd
1a77079
```

```
KAKAO_CLIENT_SECRET=ShFzjcUFJlp4Sm1fi4pEF
NeqnwHQ2rW4
```

```
AWS_ACCESS_KEY=xxxxxxxxxxxxxxxxxxxxxxxx
```

```
AWS_SECRET_KEY=xxxxxxxxxxxxxxxxxxxxxxxx
```

```
AWS_REGION=ap-northeast-2
```

```
AWS_S3_BUCKET_NAME=ssafy-c204-lol
```

```
AWS_SQS_QUEUE_NAME=JudgeQueue.fifo
```

```
LIVEKIT_API_KEY=APIMYrd8uLXWcPm
```

```
LIVEKIT_API_SECRET=GRfsrr0HbiQcF75xV3JJXPF
SGVXaRweKI61Ppe2h93RB
```

```
PROD_JUDGE_SERVER_URL=http://13.124.243.106:
8080
```

```
VITE_WEB_RTC_BASE_URL=wss://lol-demh9a97.liv
ekit.cloud
```

▼ Backend-Judge-SERVER

```
c204-be-judge/
└── .env # 파일 위치
```

```
SERVER_PORT=8080
```

```
MYSQL_LOCAL_HOST=localhost
```

```
MYSQL_LOCAL_PORT=3307
```

```
MYSQL_LOCAL_USERNAME=root
```

```
MYSQL_LOCAL_PASSWORD=1234
```

```
MYSQL_TEST_HOST=i13C204.p.ssafy.io
```

```
MYSQL_TEST_PORT=3307
```

```
MYSQL_TEST_USERNAME=root
```

```
MYSQL_TEST_PASSWORD=1q2w3e4r!
```

```
MYSQL_PROD_HOST=i13C204.p.ssafy.io
```

```
MYSQL_PROD_PORT=3306
```

```
MYSQL_PROD_USERNAME=root
```

```
MYSQL_PROD_PASSWORD=1q2w3e4r!
```

```
MYSQL_DATABASE=lol
```

```
AWS_ACCESS_KEY=xxxxxxxxxxxxxxxxxxxxxxxx
```

```
AWS_SECRET_KEY=xxxxxxxxxxxxxxxxxxxxxxxx
```

```
AWS_REGION=ap-northeast-2
```

```
AWS_SQS_QUEUE_NAME=JudgeQueue.fifo

LOCAL_API_SERVER_URL=http://localhost:8081
PROD_API_SERVER_URL=http://i13C204.p.ssafy.io:8081

JUDGE_RESULT_API_PATH=/api/v1/problems/judge-result/
```

배포 시 특이사항 및 배포 방법

1. 알고리즘 채점 서버는 별도의 EC2 인스턴스에서 배포되었음
2. 알고리즘 채점 서버의 원활한 동작을 위해 지정된 위치에 아래 유형의 디렉토리를 미리 생성하여야 함.
 - 사용자의 소스코드 (/home/ubuntu/sourceCode)
 - 채점에 사용될 테스트케이스 (/home/ubuntu/testcases)
 - 메타 파일 (/home/ubuntu/c204-be-judge/meta)
3. 알고리즘 채점 서버는 ubuntu 환경에서만 동작함
4. AWS 서비스 환경은 사전에 구성되어야 하며, 다음 리소스를 포함합니다.
 - S3, SQS

```
#MySQL
docker pull mysql:8.0.42

docker run -d \
  --name mysql \
  -e MYSQL_ROOT_PASSWORD=1q2w3e4r! \
  -e MYSQL_DATABASE=lol \
  -p 3306:3306 \
  mysql:8.0.42

#FRONT-END
docker pull nove1080/c204-fe

docker run -d \
  --name c204-fe \
  -p 80:80 \
  -p 443:443 \
  nove1080/c204-fe:latest

#BACK-END: API-SERVER
docker pull nove1080/c204-be-api

docker run -d \
  --name c204-be-api \
  -p 8081:8081 \
  nove1080/c204-be-api:latest

#BACK-END: JUDGE-SERVER
sudo apt update
sudo apt install openjdk-17-jdk
sudo apt install g++
```

```

sudo mkdir -p /etc/apt/keyrings
curl https://www.ucw.cz/isolate/debian/signing-key.asc | sudo tee /etc/apt/keyrings/isolate.asc > /dev/null
echo "deb [arch=amd64 signed-by=/etc/apt/keyrings/isolate.asc] http://www.ucw.cz/isolate/debian/ bookworm-isolat
e main" | sudo tee /etc/apt/sources.list.d/isolate.list
sudo apt update
sudo apt install isolate

git clone https://lab.ssafy.com/skwpqjq/c204-be-judge.git
cd c204-be-judge
cp <env 파일 경로> .env

./gradlew clean build

java -jar -Dspring.profiles.active=prod c204-be-judge-0.0.1-SNAPSHOT.jar

```

Nginx 설정

```

upstream backend_api {
    server i13c204.p.ssafy.io:8081;
}

server {
    listen            80;
    server_name       i13c204.p.ssafy.io;

    location / {
        return 301 https://$host$request_uri;
    }

    error_log         /var/log/nginx/error.log;
    access_log         /var/log/nginx/access.log;
}

server {
    listen            443 ssl;
    server_name       i13c204.p.ssafy.io;

    ssl_certificate /etc/letsencrypt/live/i13c204.p.ssafy.io/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/i13c204.p.ssafy.io/privkey.pem;
    include /etc/letsencrypt/options-ssl-nginx.conf;
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;

    location / {
        root          /usr/share/nginx/html;
        index          index.html;

        try_files $uri $uri/ /index.html;
    }

    location /ws {
        proxy_pass      http://backend_api;

        proxy_http_version 1.1;
        proxy_set_header  Host $host;
    }
}

```

```

        proxy_set_header    Upgrade $http_upgrade;
        proxy_set_header    Connection "Upgrade";
    }

    location /api {
        proxy_pass            http://backend_api;

        proxy_set_header     Host $host;
        proxy_set_header     X-Real-IP $remote_addr;
        proxy_set_header     X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header     X-Forwarded-Proto $scheme;
        proxy_set_header     Upgrade $http_upgrade;
        proxy_set_header     Connection "upgrade";

        add_header           'Access-Control-Allow-Origin' 'https://i13c204.p.ssafy.io' always;
        add_header           'Access-Control-Allow-Headers' 'Content-Type, Authorization' always;
        add_header           'Access-Control-Allow-Credentials' 'true' always;

        if ($request_method = 'OPTIONS') {
            add_header       'Access-Control-Allow-Origin' 'https://i13c204.p.ssafy.io';
            add_header       'Access-Control-Allow-Methods' 'GET, POST, DELETE, PATCH, OPTIONS';
            add_header       'Access-Control-Allow-Headers' 'Content-Type, Authorization';
            add_header       'Access-Control-Allow-Credentials' 'true';

            return 204;
        }

        proxy_redirect      off;
    }
}

```

빌드 방법

1. 백엔드: API-Server

```

git clone https://lab.ssafy.com/skwpqjq/c204-be-api.git

cd c204-be-api
cp <env 파일 경로> .env

docker build -t <dockerhub_id>/<image_name>:<tag> .
docker run -d \
    --name <container_name> \
    -p 8081:8081 \
    <dockerhub_id>/<image_name>:<tag>

```

2. 백엔드: Judge-Server

```

git clone https://lab.ssafy.com/skwpqjq/c204-be-judge.git

cd c204-be-judge
cp <env 파일 경로> .env

./gradlew clean build

```

```
java -jar -Dspring.profiles.active=prod c204-be-judge-0.0.1-SNAPSHOT.jar
```

3. 프론트엔드

```
git clone https://lab.ssafy.com/rudrn0110/c204-fe.git

cd c204-fe/fe
cp <env 파일 경로> .env.production

docker build -t <dockerhub_id>/<image_name>:<tag> .
docker run -d \
  --name <container_name> \
  -p 80:80 \
  -p 443:443 \
  <dockerhub_id>/<image_name>:<tag>
```

DB 접속 정보

URL: jdbc:mysql://i13C204.p.ssafy.io:3306/lol?serverTimezone=Asia/Seoul&characterEncoding=utf8
User: root
Password: 1q2w3e4r!

더미 데이터 파일

[data_v250815_v2.sql](#)